



Project Final Report

Project Title: PEARLS AQIPREDICTOR

SUBMITTED BY: Syed Huzaifa Ali

Email: hh7408406@gmail.com

Live Application:

<https://aqiproject-6rqnaxgtnz7bbizdkr2ylt.streamlit.app/>

Internship Mentor: Muhammad Mobeen

Table of Contents

CONTENTS	PAGE NUMBER
1- Acknowledgements	2
2- Abstract	3
3- Background	4
4- Problem Statement	5
5- Concepts and Theory Used	6-8
6- Methodology	9-11
7- Problems Faced and their Solutions	12
8- CI/CD Pipelines	13
9- Pipeline Visualization and Architecture	14
10- Streamlit Web Application	15-18
11- Conclusion	19

1.Acknowledgements

I would like to thank my mentor Muhammad Mobeen at 10Pearls for his guidance and patience throughout this project. His feedback at every stage from identifying data leakage to suggesting better feature engineering approaches was invaluable. I also want to acknowledge the open-source communities behind OpenWeather API, Open-Meteo, Hopsworks, Streamlit, and GitHub Actions, without which this project would not have been technically or financially feasible. Special thanks to my fellow cohort members whose discussions in the group channel helped me understand common problems faster and validate that certain issues were platform-level rather than code-level.

2. Abstract

This project presents a complete, production-grade Air Quality Index prediction system built for Karachi, Pakistan. The system forecasts pollution levels for the next 72 hours through an interactive web application that updates automatically every hour without any manual intervention. Raw pollution and weather data is fetched hourly from OpenWeather and Open-Meteo APIs, stored in a Hopsworks Feature Store, transformed into 18 engineered features, and used to train three machine learning models daily. A recursive forecasting loop extends this to 72 hours, and the results are displayed on a Streamlit dashboard with pages for current conditions, forecast, model performance, exploratory data analysis, SHAP explainability, and project information. The entire pipeline runs automatically via GitHub Actions CI/CD workflows on a fixed schedule with no human intervention required after initial setup.

3. Background

Karachi is Pakistan's largest city with over 20 million residents, and air pollution has become one of its most serious public health challenges. PM2.5 levels regularly exceed safe limits during the winter months of December through March, when temperature inversions trap pollutants near the ground. People with respiratory conditions, the elderly, and children are particularly vulnerable, and they need advance warning not just current readings to make informed decisions about outdoor activities. Traditional air quality services only report current conditions. There is no freely available service that provides 72-hour AQI forecasts specifically for Karachi.

This project was built to address that gap by combining machine learning with automated data pipelines and a publicly accessible web dashboard. The project also serves as a demonstration of end-to-end MLOPs practices, including feature engineering, feature stores, model registries, CI/CD automation, and model explainability all using free tools that could be replicated by anyone with a GitHub account and API keys.

4. Problem Statement

The central question this project tries to answer is: **How can we automatically predict the next 72 hours of air quality for Karachi using real-time data, with no manual intervention once the system is set up?**

This question breaks down into several smaller challenges. First, pollution data needs to be collected continuously and stored reliably. Second, raw measurements need to be transformed into features that a machine learning model can actually learn from. Third, models need to retrain automatically as new data arrives so they do not go stale. Fourth, predictions need to be served through a user-friendly interface that always shows fresh, accurate forecasts. And fifth, the entire system needs to run on free infrastructure so it is sustainable beyond the internship period.

5. Concepts and Theory Used

This project uses the following core ideas:

5.1 Air Quality Index (AQI)

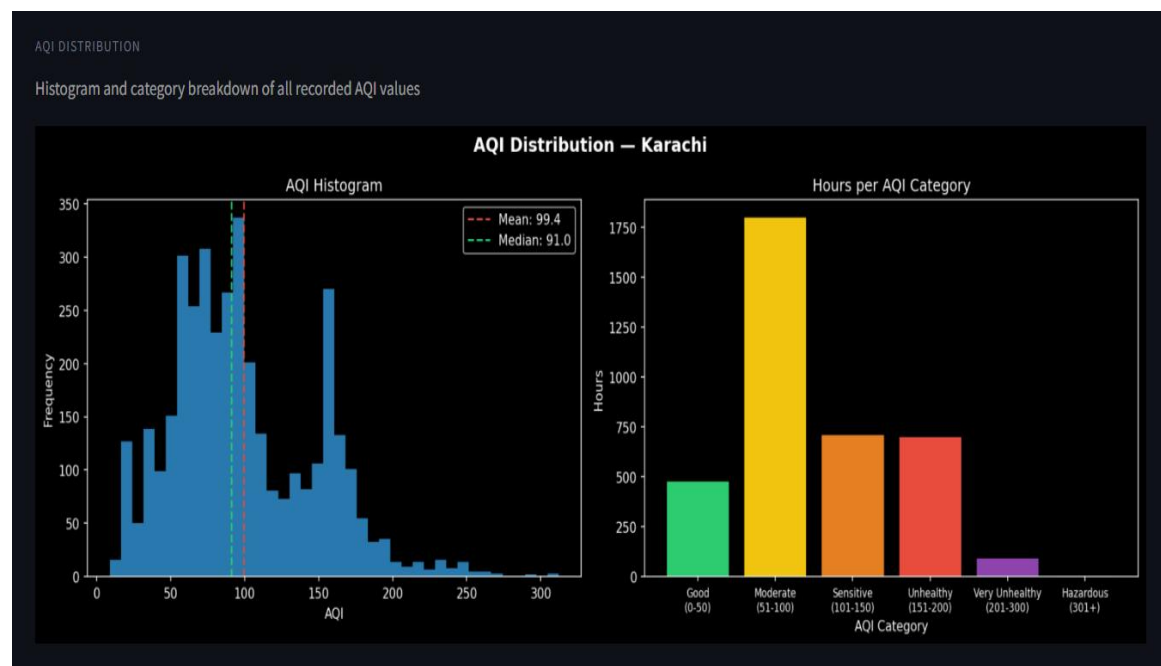
AQI is a standardized indicator used to measure air pollution levels and their impact on human health.

This project uses:

- PM2.5 concentration as the primary pollutant for prediction.
- US EPA AQI breakpoints to calculate AQI values.
- AQI categories ranging from Good to Hazardous.
- AQI computation from raw PM2.5 measurements instead of the OpenWeather AQI scale.

Key Finding

- OpenWeather provides AQI on a 1–5 scale, which was not suitable for machine learning.
- EPA AQI was calculated directly from PM2.5 values to obtain realistic pollution levels.



5.2 Feature Engineering

Feature engineering transforms raw pollution and weather data into meaningful inputs for machine learning models.

Features created include:

Time Features

- Hour of day
- Day of week
- Month

Lag Features

- PM2.5 Lag 1 Hour
- PM2.5 Lag 3 Hours

Rolling Features

- 3-Hour Rolling Average
- 6-Hour Rolling Average

Weather Features

- Temperature Lag Features
- Humidity Lag Features
- Wind Speed Lag Features

Other Features

- Pollution Change Rate
- Weekend Indicator

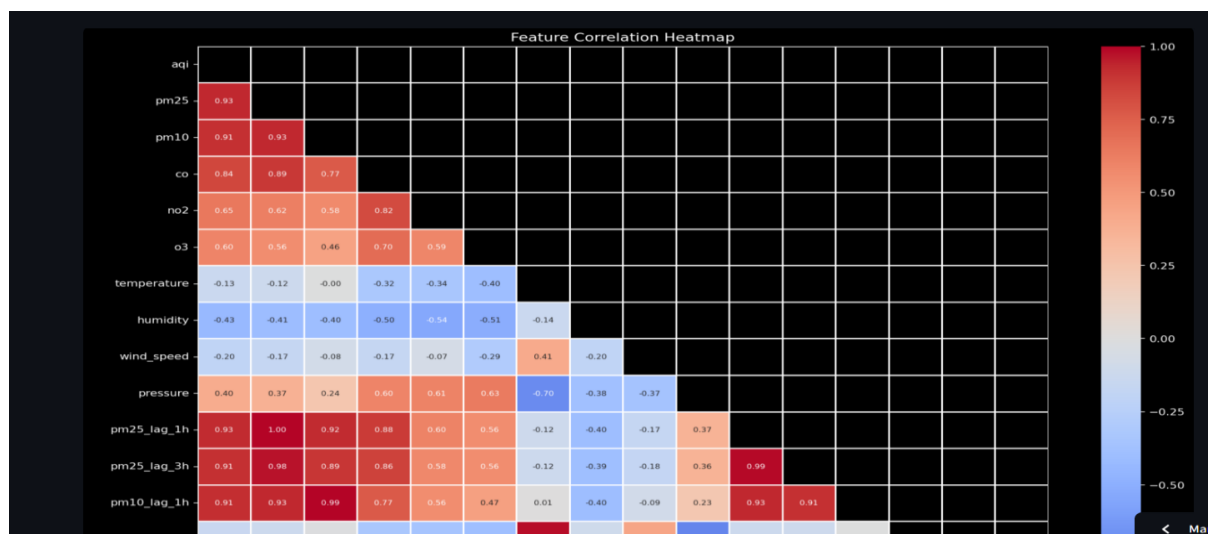
A total of 18 engineered features were generated automatically before model training.

aqi_features version 2

Q Find feature type Primary Keys Partition Keys Event Time sort by name

33 out of 33 features displayed - last 50 rows

	aqi	aqi_next_hour	co	day	day_of_week	hour	humidity	humidity_lag_1h	humidity_rol_3h	is_weekend	month	nh3	no	no2	o3	pm10	pm25
35																	
36	138	138.0	318.52	29	3	17	41.0	38.0	37.666666666666664	0	1	1.13	0.0	1.73	143.34	106.52	103
37	65	69.0	179.73	24	5	17	43.0	41.0	41.0	1	1	1.21	0.0	1.32	114.38	67.59	60
38	87	94.0	131.43	14	5	23	72.0	67.0	69.33333333333333	1	3	0.0	0.0	0.11	95.58	80.72	69
39	62	60.0	118.69	15	2	16	82.0	81.0	80.66666666666667	0	4	0.0	0.0	0.2	106.98	53.52	62
40	89	91.0	100.11	22	2	13	78.0	74.0	74.0	0	4	0.0	0.01	0.08	50.47	107.45	106
41	152	153.0	315.78	30	4	6	35.0	42.0	42.333333333333336	0	1	1.22	0.27	2.27	127.77	142.03	138
42	114	113.0	175.12	8	6	20	89.0	90.0	90.0	1	2	0.0	0.0	0.09	97.7	89.33	89
43	131	133.0	264.34	12	0	5	39.0	44.0	44.0	0	1	1.82	0.18	2.02	126.46	90.36	90
44	150	147.0	124.75	13	4	21	71.0	70.0	69.0	0	3	0.05	0.0	0.18	97.66	159.88	156
45	85	83.0	265.22	25	6	0	47.0	45.0	45.666666666666664	1	1	1.9	0.0	1.09	117.22	89.49	93
46	138	141.0	307.05	30	4	2	55.0	55.0	55.0	0	1	1.37	0.0	1.73	118.18	125.44	120
47	151	148.0	275.97	9	1	17	37.0	36.0	36.333333333333336	0	12	0.0	0.0	1.66	146.12	110.97	117



5.3 Machine Learning Models

Several regression models were trained and compared to predict future AQI values.

Models used:

- Ridge Regression
- Random Forest Regressor
- XGBoost Regressor

Model Evaluation Metrics

The best model was selected using:

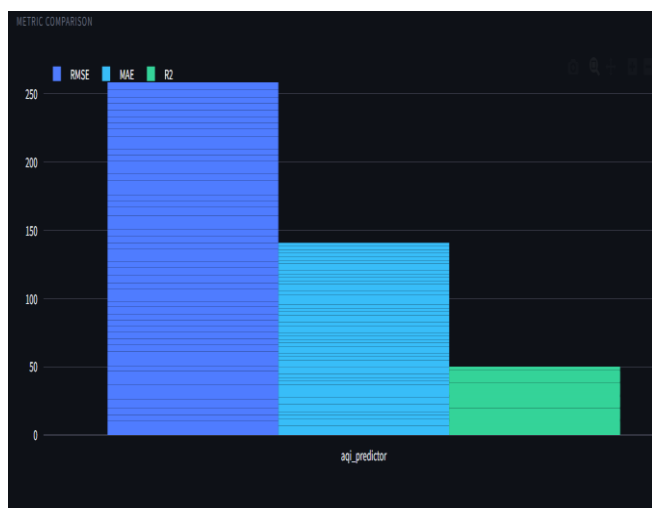
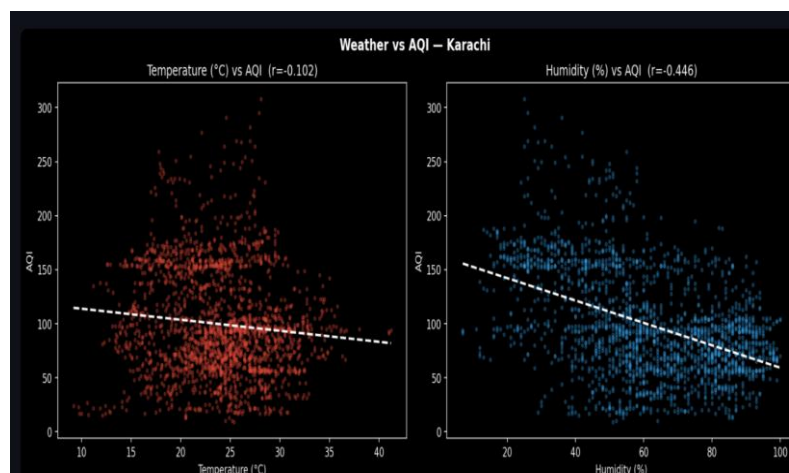
- RMSE (Root Mean Square Error)
- MAE (Mean Absolute Error)
- R^2 Score

72-Hour Forecasting

The model predicts AQI six hours ahead.

To generate a 72-hour forecast:

- Predict AQI for T+6 hours.
- Use the prediction as input for the next step
- Predict T+12 hours.
- Continue recursively until T+72 hours.



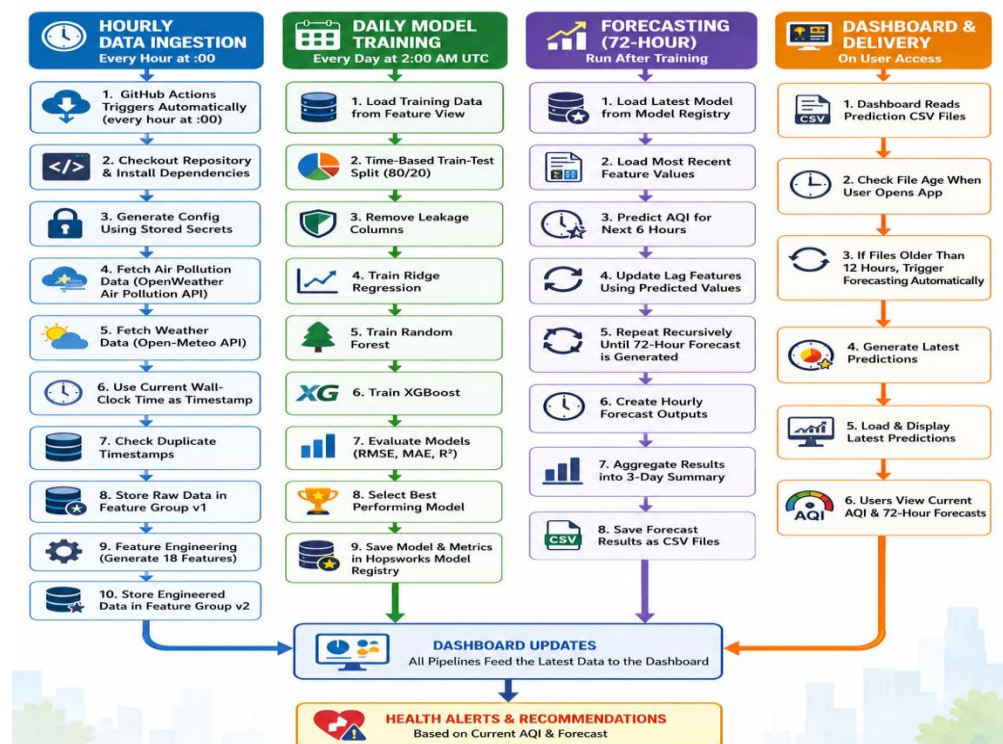
5.4 MLOps Concepts

The project follows modern MLOps practices to automate the complete forecasting workflow.

Feature Store, Model Registry, CI/CD Automation:

1. Hopsworks Feature Store is used to store engineered features.
2. Provides centralized and version-controlled feature management.
3. Stores trained model versions.
4. Tracks evaluation metrics and model lineage.
5. GitHub Actions automates data collection.
6. Daily model retraining is performed automatically.
7. Forecast generation and deployment are fully automated.

6. Methodology



Stage 1: Hourly Data Ingestion Pipeline

Files:

- `.github/workflows/feature_pipeline.yml`
- `src/data_pipeline/ingest_hourly.py`
- `src/data_pipeline/feature_engineering.py`

- Collects the latest air pollution and weather data every hour.
- Stores new observations in the Feature Store.
- Automatically updates engineered features.

How it works:

The hourly data ingestion process is fully automated using GitHub Actions, which runs at minute 0 of every hour. During each execution, the repository is checked out and all required dependencies are installed. A configuration file is then generated using securely stored secrets. The system fetches air pollution data from the OpenWeather Air Pollution API and weather information from the Open-Meteo API. The current wall-clock time is used as the timestamp for the collected records. Before storing the data, the system checks for duplicate timestamps to prevent redundant entries. The validated raw data is then stored in Feature Group v1. Afterward, a feature engineering pipeline generates 18 engineered features from the raw data, and the resulting transformed dataset is stored in Feature Group v2.

Hourly Feature Pipeline	main	Today at 8:46 AM	1m 52s
Hourly Feature Pipeline	main	Today at 4:21 AM	1m 35s
Hourly Feature Pipeline	main	Today at 2:56 AM	1m 32s
Hourly Feature Pipeline	main	Today at 12:25 AM	1m 38s

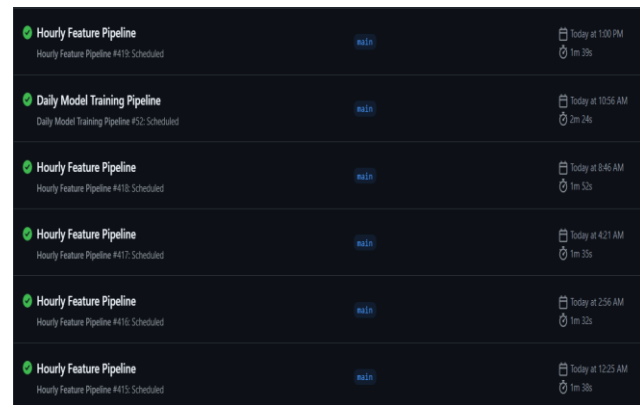
Stage 2: Automated Model Training Pipeline

Files

- `.github/workflows/training_pipeline.yml`
- `src/training_pipeline/train_model.py`
- Retrains machine learning models every day.
- Compares model performance.
- Stores the best model in the Model Registry.

How it works:

The model training pipeline runs automatically every day at 2:00 AM UTC using GitHub Actions. During each execution, the training dataset is retrieved from the Feature View, which combines the engineered features and target values required for model development. The data is then divided using an 80/20 time-based train-test split to preserve the chronological order of observations and simulate real-world forecasting conditions. Before training, any leakage columns that could reveal future information are removed to ensure fair model evaluation. Three machine learning models Ridge Regression, Random Forest, and XGBoost are then trained on the prepared dataset. After training, the models are evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the R^2 score. The model that achieves the best overall performance is automatically selected as the champion model. Finally, the selected model along with its evaluation metrics is registered and stored in the Hopsworks Model Registry for versioning, tracking, and deployment.



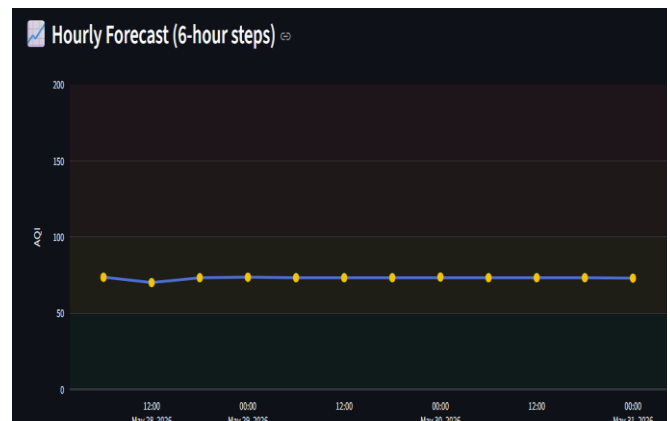
Stage 3: Forecast Generation Pipeline

Files

- `src/forecasting/forecast.py`
- Generates AQI forecasts for the next 72 hours
- Produces daily forecast summaries.

How it works:

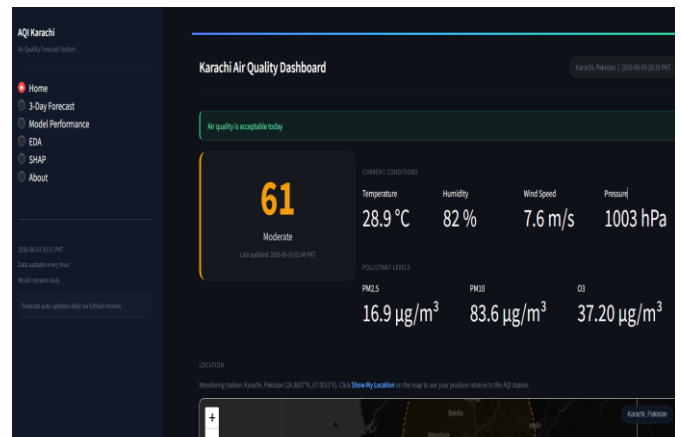
The forecasting system begins by loading the latest trained model from the Model Registry, along with the most recent feature values required for inference. Using this data, it predicts AQI values for the next 6 hours. These predictions are then fed back into the system to update lag features, allowing the model to incorporate its own outputs into future steps. This recursive process is repeated until a full 72-hour forecast horizon is generated. After generating all hourly predictions, the results are structured into hourly forecast outputs. These hourly values are then aggregated to produce a 3-day forecast summary. Finally, both the detailed forecasts and summarized results are saved as CSV files for further analysis and reporting.



Stage 4: Dashboard Update Mechanism Files

- `app.py`
 - `predictions/hourly_forecast.csv`
 - `predictions/daily_summary.csv`
- Displays the latest AQI forecasts to users.
 - Automatically refreshes outdated predictions.

How it works:



The dashboard operates by reading prediction CSV files to display AQI forecasts. Whenever a user opens the application, it first checks the age of the existing prediction files. If the files are older than 12 hours, the system automatically triggers the forecasting pipeline to generate updated results. Once refreshed, the latest predictions are loaded and displayed on the dashboard. Users can then view both the current AQI values and the 72-hour forecast in a unified interface for real-time monitoring and short-term air quality planning.

Stage 5: End-to-End Automation

Technologies Used

- GitHub Actions
- Hopworks Feature Store
- Hopworks Model Registry
- OpenWeather API
- Open-Meteo API
- Streamlit Cloud

Complete Workflow

1. Collect hourly pollution and weather data.
2. Store raw data in Feature Store.
3. Generate engineered features.
4. Retrain models daily.
5. Save best model to Model Registry.
6. Generate 72-hour AQI forecasts.
7. Save prediction files.
8. Update Streamlit dashboard automatically.

7. Problems Faced and Their Solutions

Problem 1 — AQI Constant at 3, RMSE = 0

During initial training, the model achieved an unrealistically perfect RMSE of 0.0000 and consistently predicted AQI = 3. Investigation showed that the OpenWeather Air Pollution API returns AQI on a **1–5 European scale** (record["main"]["aqi"]), not the EPA 0–500 scale. Since Karachi frequently reported a constant value of 3, the dataset had no variation, causing the model to learn a trivial constant mapping.

Solution:

AQI was recalculated using **EPA PM2.5 breakpoint formulas**, converting raw pollutant concentrations into realistic AQI values (range: ~9–312). This introduced true variability across time and significantly improved model learning.

Problem 2 — Data Leakage and Unreliable Low RMSE

Even after fixing AQI scaling, RMSE remained artificially low (~3.48). The model was still using raw pollutant features (CO, NO, NO₂, O₃, PM10, PM2.5), which directly determine AQI, causing **target leakage**. Additionally, predicting aqi.shift(-1) made the task too easy due to minimal short-term variation.

Solution:

- Removed all pollutant variables from features (added to leakage list)
- Increased prediction horizon to aqi.shift(-6) (6-hour forecasting)

Problem 3 — Major Hopsworks Spark Materialization Failure (Platform Issue)

Starting May 21, 2026, a critical issue was observed in the Hopsworks platform where Spark materialization jobs began failing system-wide. Although data ingestion into the Feature Store was functioning correctly and feature records were being successfully written, the materialization process consistently failed to execute.

Jobs remained stuck in a “**submitted**” state or failed during scheduling without progressing to execution. Kubernetes logs indicated the following error:

“FailedScheduling: 0/23 nodes available, insufficient CPU”

Investigation and cross-validation with other interneer confirmed that this was not related to any code-level or pipeline configuration issue. The same behavior was observed across multiple independent projects, indicating a **platform-wide infrastructure limitation or resource exhaustion within the Hopsworks Spark cluster**.

Resolution — System Restored

The Hopsworks materialization issue has now been fully resolved. Spark materialization jobs are executing successfully, and all feature pipelines are completing end-to-end without interruption.

8. CI/CD PIPELINES

The project uses two automated GitHub Actions workflows. The hourly feature pipeline runs at the start of every hour and takes approximately three to four minutes to complete. It fetches new pollution and weather data, performs the duplicate timestamp check, inserts the new row into the feature store, and runs feature engineering to update the engineered features. The daily training pipeline runs at 2:00 AM UTC and takes approximately five to seven minutes. It retrains all three models on the full accumulated dataset, selects the best one by RMSE, generates the 72-hour recursive forecast, and commits the fresh prediction CSV files to the repository.

Both workflows use GitHub Secrets to store sensitive credentials including the Hopsworks API key, OpenWeather API key, and email app password. The config.yaml file is generated fresh on every workflow run from these secrets rather than being stored in the repository, which ensures no sensitive information is ever committed to version control.

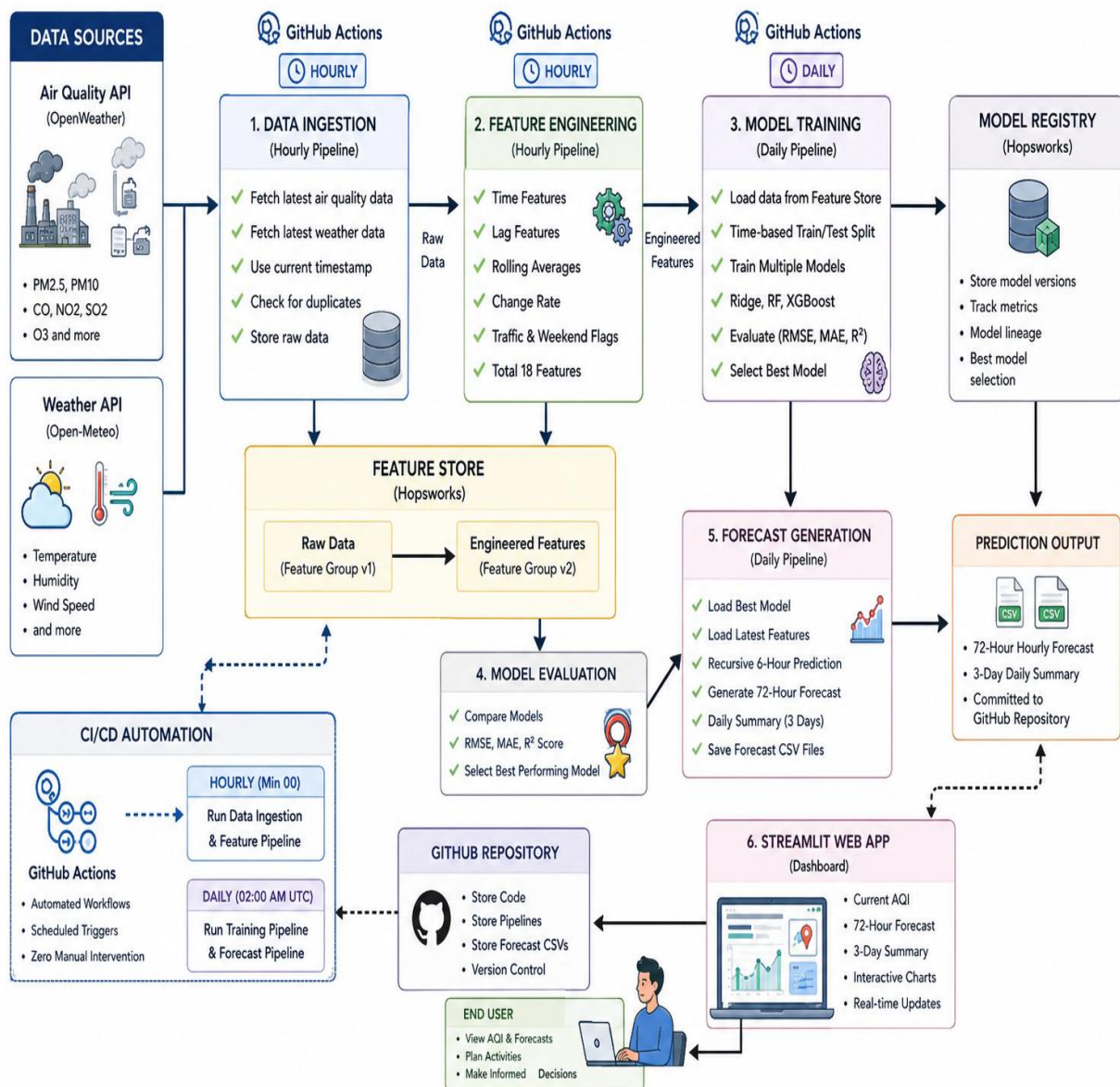
The screenshot displays the GitHub Actions interface for a repository. On the left, the 'Actions' tab is selected, showing a list of workflows: 'All workflows', 'Daily Model Training Pipeline', and 'Hourly Feature Pipeline'. The 'Hourly Feature Pipeline' is currently selected. The main area shows a list of workflow runs, each with a green checkmark indicating success. The runs are listed in descending order of time, with the most recent run at the top. Each run entry includes the workflow name, a link to the run details, the branch (main), and the time taken to complete the run. The runs are scheduled at regular intervals, with the 'Hourly Feature Pipeline' running every hour and the 'Daily Model Training Pipeline' running at 2:00 AM UTC. The interface also shows a 'New workflow' button and a 'Management' section with links to 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'.

Workflow Name	Run ID	Status	Branch	Time	Duration
Hourly Feature Pipeline	Hourly Feature Pipeline #464: Scheduled	Success	main	Jun 2, 11:40 PM GMT+5	7m 22s
Hourly Feature Pipeline	Hourly Feature Pipeline #463: Scheduled	Success	main	Jun 2, 7:54 PM GMT+5	8m 1s
Hourly Feature Pipeline	Hourly Feature Pipeline #462: Scheduled	Success	main	Jun 2, 3:26 PM GMT+5	7m 29s
Daily Model Training Pipeline	Daily Model Training Pipeline #57: Scheduled	Success	main	Jun 2, 11:24 AM GMT+5	2m 27s
Hourly Feature Pipeline	Hourly Feature Pipeline #461: Manually run by HuzaiFaali1020	Success	main	Jun 2, 11:22 AM GMT+5	7m 46s
Hourly Feature Pipeline	Hourly Feature Pipeline #460: Manually run by HuzaiFaali1020	Success	main	Jun 2, 10:57 AM GMT+5	7m 34s
Hourly Feature Pipeline	Hourly Feature Pipeline #458: Scheduled	Success	main	Jun 2, 5:14 AM GMT+5	8m 5s
Hourly Feature Pipeline	Hourly Feature Pipeline #457: Scheduled	Success	main	Jun 2, 3:11 AM GMT+5	8m 5s
Hourly Feature Pipeline	Hourly Feature Pipeline #456: Scheduled	Success	main	Jun 1, 11:38 PM GMT+5	7m 10s

9- PIPELINE VISUALIZATION AND ARCHITECTURE

My overall project is working like below, using visuals for showing better visibility which shows overall architecture.

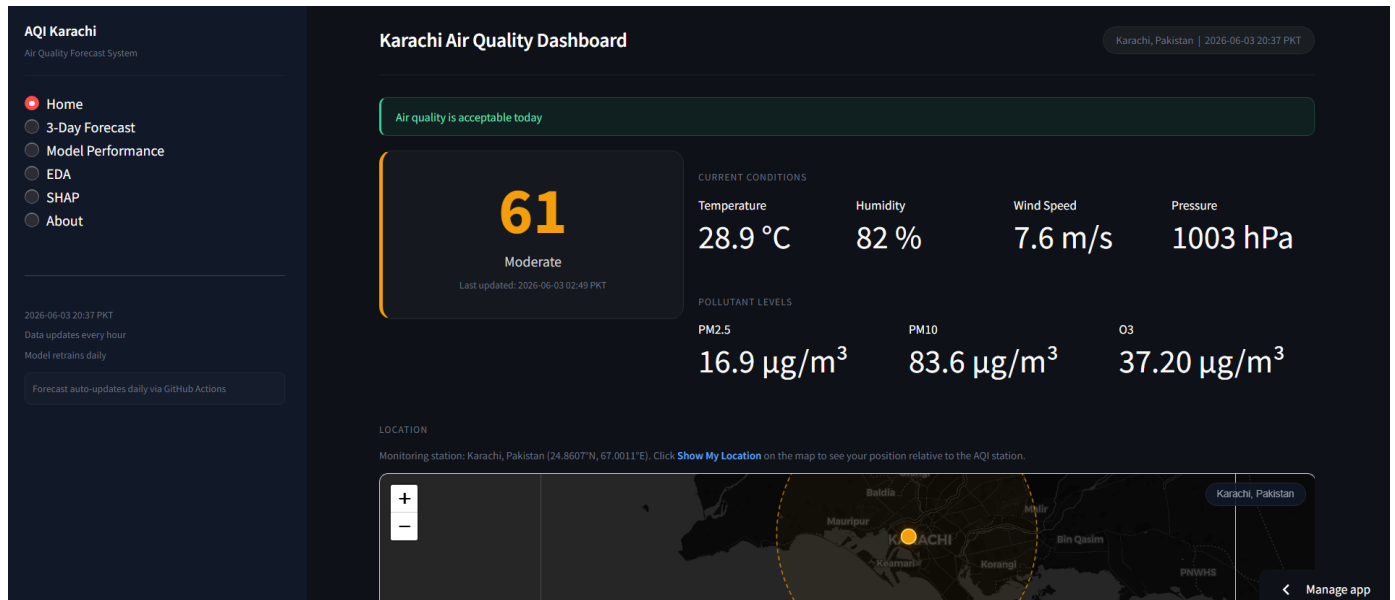
The overall data flow moves through four major stages. In the first stage, raw data enters the system from the OpenWeather and Open-Meteo APIs every hour via the GitHub Actions trigger. In the second stage, this data is cleaned, transformed, and engineered into 18 features and stored in the Hopworks Feature Store. In the third stage, the daily trigger reads all accumulated data from the Feature Store, trains three models, evaluates them against a held-out test set, and saves the best one to the Hopworks Model Registry with full lineage tracking. In the fourth stage, the forecast script loads the latest model, generates recursive predictions for the next 72 hours, saves the results as CSV files, and the Streamlit dashboard serves these predictions to end users via a live web application.



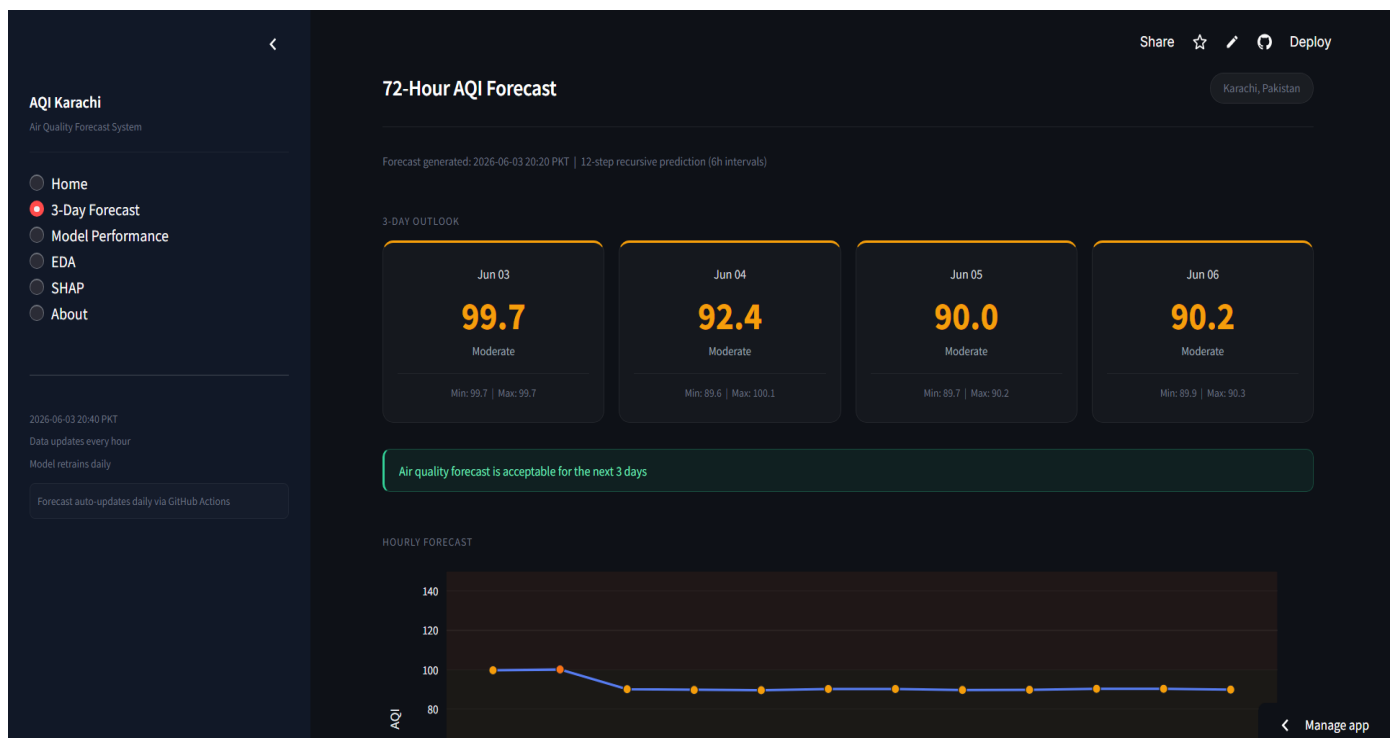
10- STREAMLIT WEB APPLICATION

There is total 6 tabs included:

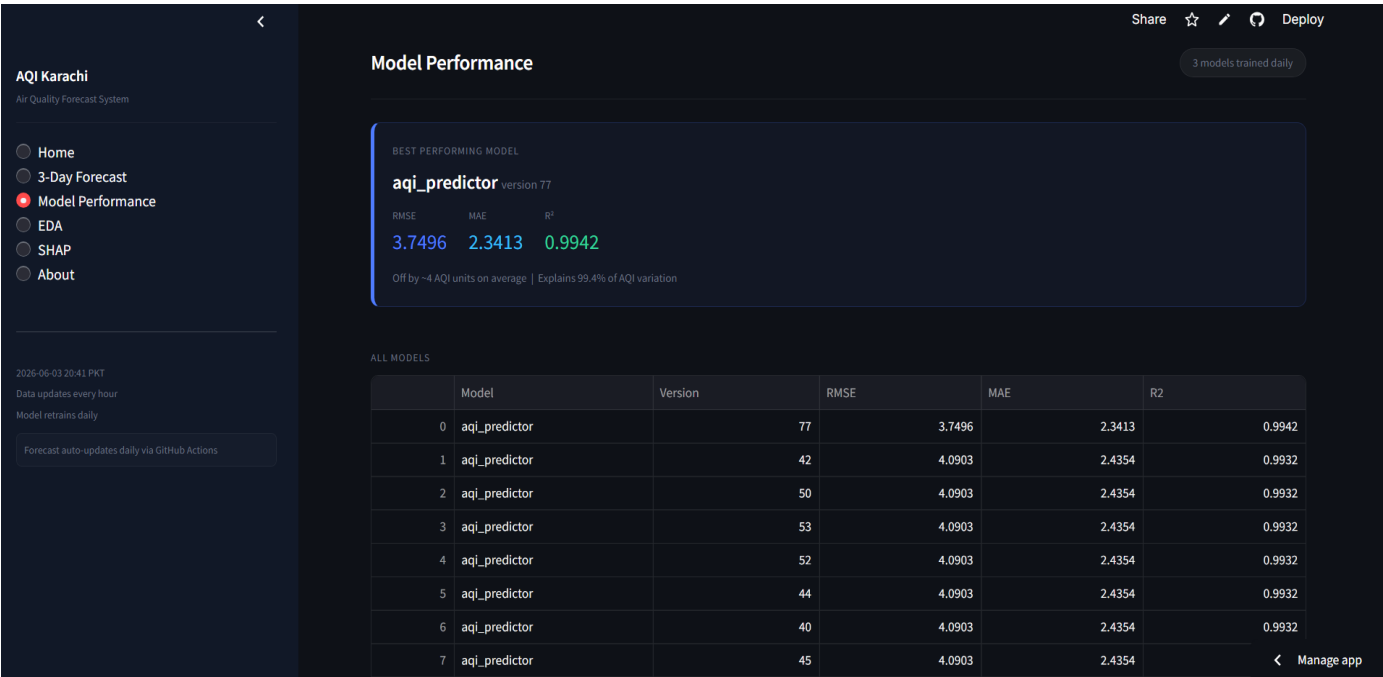
The Home page displays the current AQI as a large color-coded value with its category label and a health alert banner. Beneath it, metric cards show temperature, humidity, wind speed, and pressure, followed by current pollutant readings for PM2.5, PM10, and ozone.



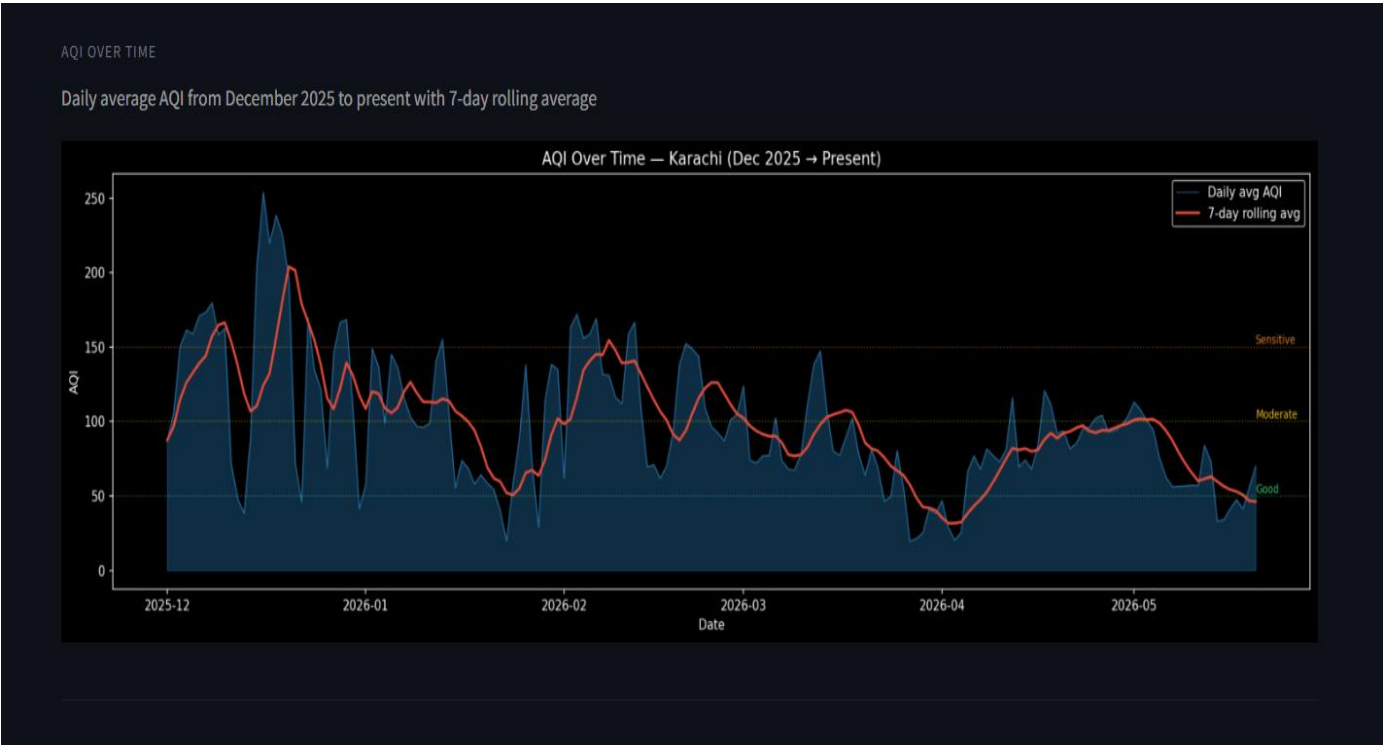
The Three-Day Forecast page presents three side-by-side cards for the next three days, each showing predicted average AQI, category, minimum, and maximum values. Below the cards, a line chart plots 12 forecast points at six-hour intervals with color-coded AQI zones, along with a health recommendation based on the average forecast.



The Model Performance page highlights the best-performing model in a dedicated card with RMSE, MAE, and R² metrics, each explained in simple AQI-related terms. A comparison table ranks all models by RMSE, and a grouped bar chart visually compares their performance.

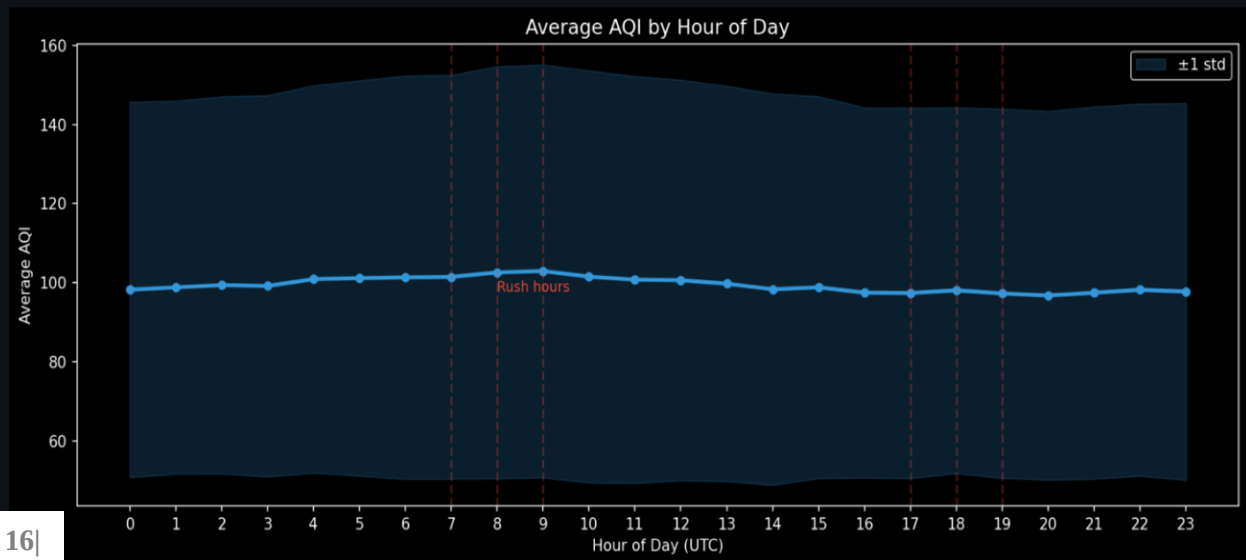


The EDA page contains seven exploratory data analysis plots generated by the EDA pipeline, each with a title and short caption describing its insights and conclusions.



HOURLY PATTERNS

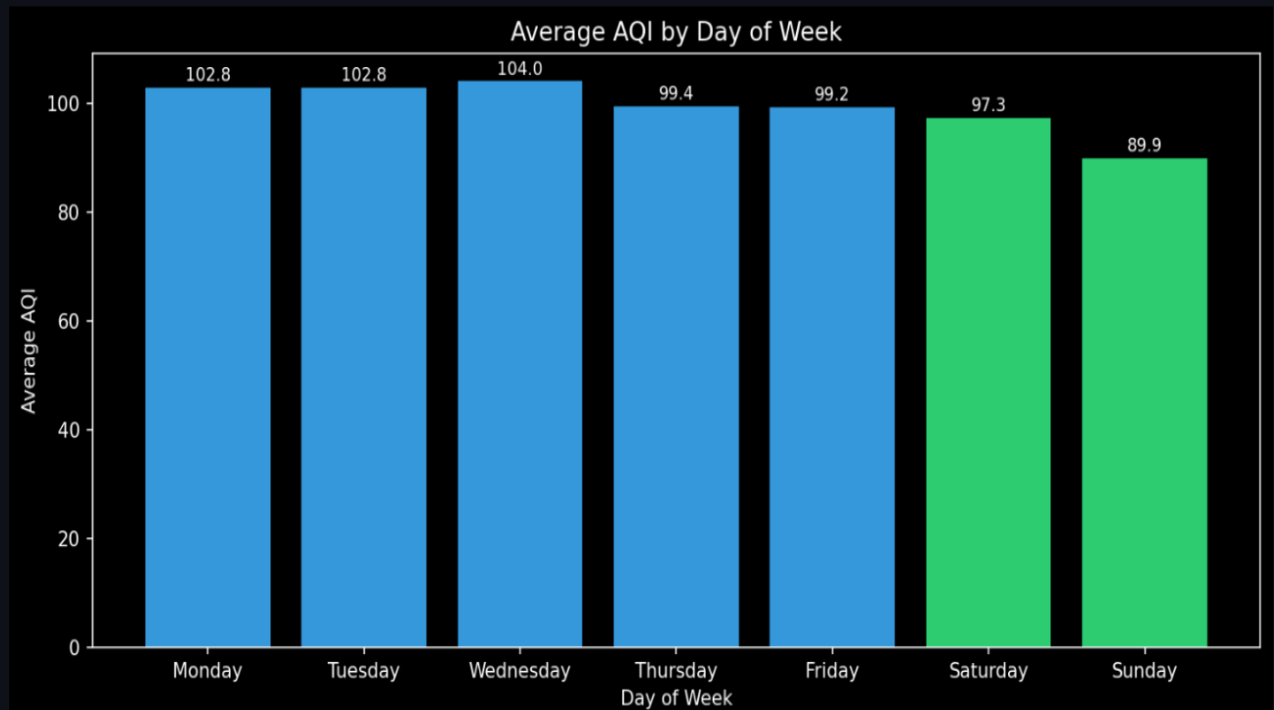
Average AQI by hour of day — identifies rush hour pollution spikes



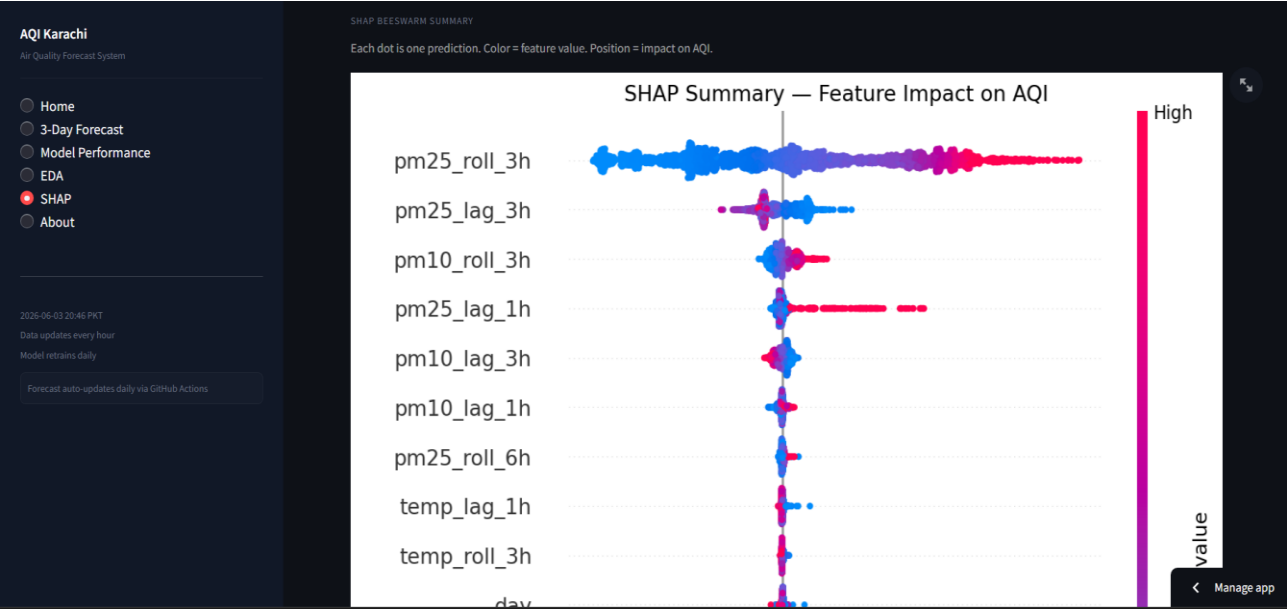
16|

WEEKLY PATTERNS

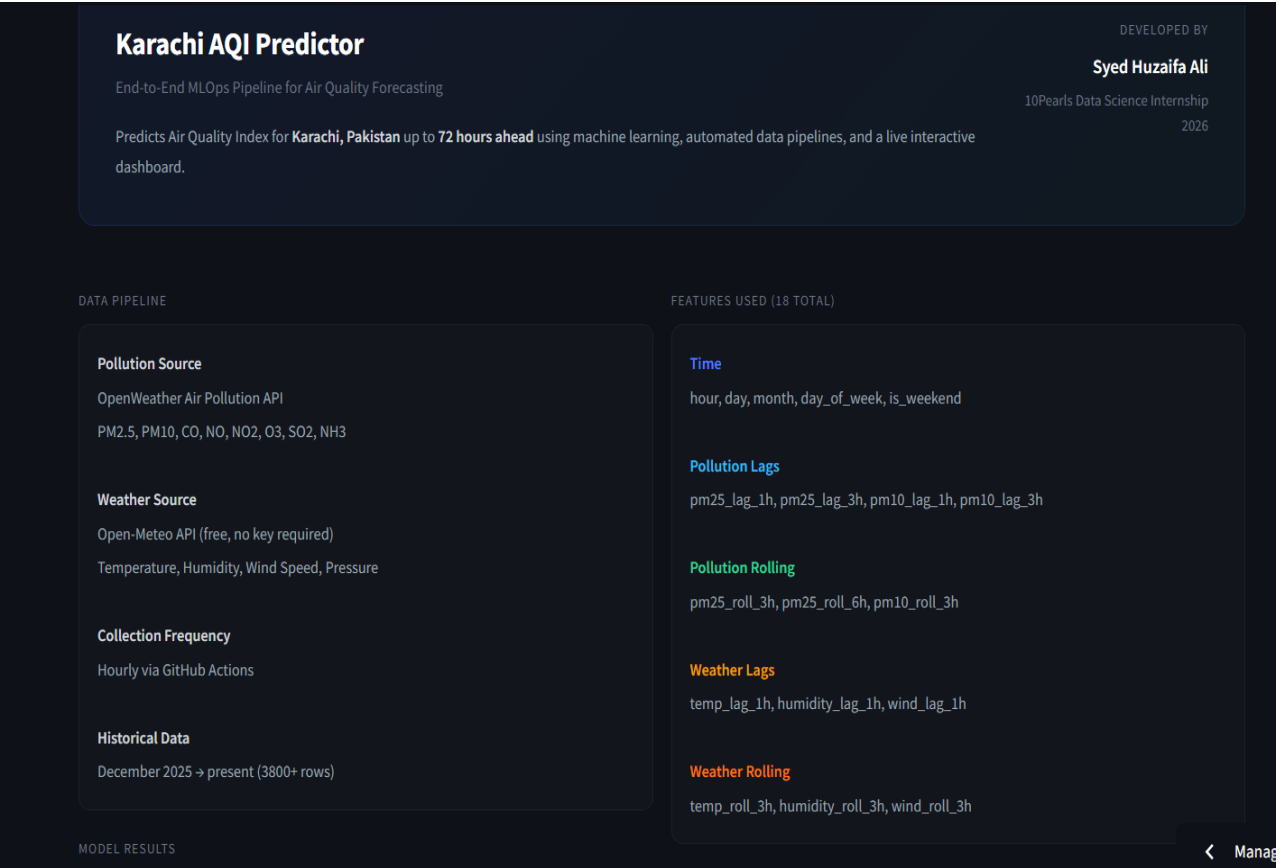
Average AQI by day of week — weekday vs weekend comparison



The SHAP page presents model interpretability results, including a feature importance bar chart, a SHAP beeswarm summary plot, and dependence plots for the top three features, showing how each feature influences predictions.



The About page describes the system architecture, lists all 18 input features, details the data sources, and includes the AQI scale reference table for user understanding.



11- CONCLUSION:

This project successfully built and deployed a complete, automated AQI prediction system for Karachi that runs continuously without human intervention. It demonstrates real-world MLOps by combining continuous data collection, automated feature engineering, daily model retraining with version tracking, feature importance explainability, and a live dashboard — all using entirely free tools and infrastructure.

What Works Well:

- a. **Full Automation** – End-to-end pipeline runs without manual intervention
- b. **Real Generalization** – Final model achieves reliable RMSE after removing multiple data leakage issues
- c. **Explainable AI** – Feature importance and SHAP analysis improve interpretability
- d. **Reliable Forecasting System** – Produces meaningful 72-hour AQI predictions
- e. **Scalable Design** – Architecture can easily extend to additional cities or regions

Key Technical Achievement:

The most significant contribution was identifying and eliminating multiple forms of data leakage that initially produced misleadingly perfect results ($RMSE \approx 0.0000$). Issues included incorrect AQI scaling, direct inclusion of target-encoded pollutant features, and overly simplistic time shifting that made prediction trivial. Each correction progressively increased model realism, culminating in a final RMSE, representing a model that genuinely generalizes to unseen data.

Infrastructure Challenge:

A major challenge was encountered in the Hopsworks eu-west Spark cluster starting May 21, 2026, where feature materialization jobs consistently failed due to “**0/23 nodes available**” scheduling errors. This indicated a **platform-side server resource limitation within Hopsworks**, causing Spark jobs to be unable to acquire compute resources for execution.

This issue was not related to any code or pipeline configuration. It was confirmed to be a **system-wide infrastructure problem**, as similar materialization failures were observed across multiple users in the cohort.

To maintain pipeline continuity, materialization dependency was bypassed so that the feature pipeline could continue executing without interruption. This ensured that data ingestion and feature generation remained fully functional despite the backend limitation.

The issue has now been **fully resolved on the Hopsworks side**, and Spark materialization jobs are functioning normally again. All feature pipelines are now running end-to-end successfully without any failures.

Submitted by Syed Huzaiifa Ali 10Pearls Data Science Internship 2026